MODEL DEVELOPMENT DOCUMENT

Personal Finance AI Agent

Capstone Project 3: Generative AI and Agentic AI (LangChain, GPT-4o-mini, Streamlit)

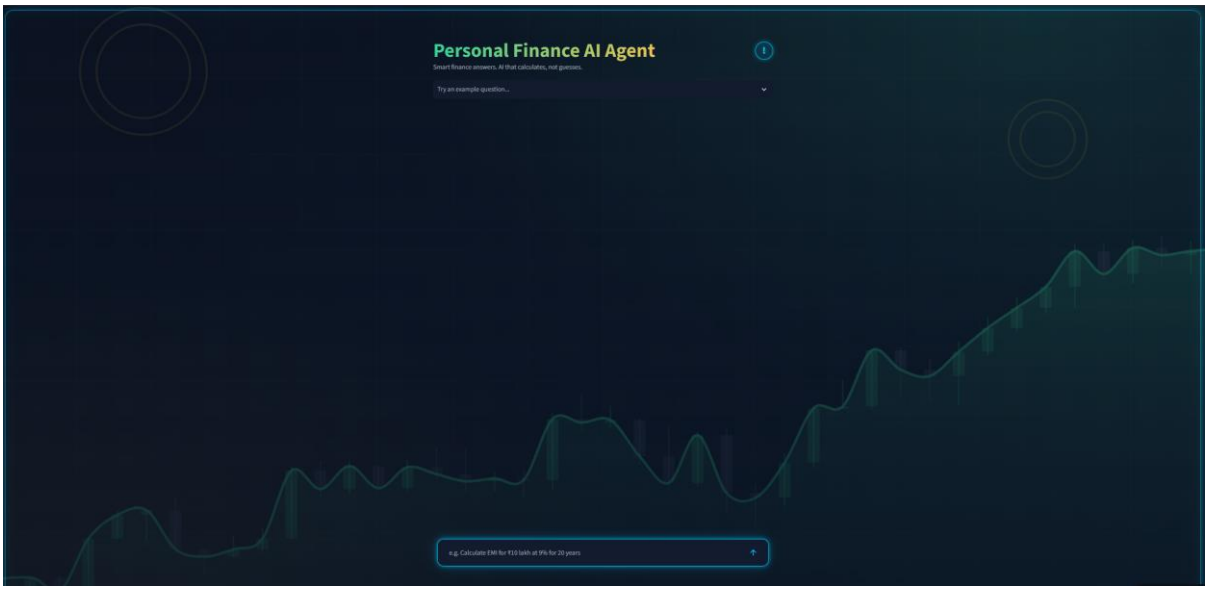


Figure 1: The deployed Personal Finance AI Agent (final Streamlit interface)

Submitted by

VINEETH M

Applied Data Science, Machine Learning and Artificial Intelligence
E & ICT Academy, IIT Guwahati
vinuvineeth616@gmail.com | +91 7560946570
October 2026

GitHub: https://github.com/Vineeth-Muraleedharan/personal-finance-ai-agent/tree/main

Streamlit app: https://personal-finance-ai-agent-by-vineeth.streamlit.app/

1. Project Overview

1.1 Project details

Project title	Personal Finance AI Agent
Programme	Applied Data Science, Machine Learning and AI, E & ICT Academy, IIT Guwahati
Capstone	Capstone 3: Generative AI and Agentic AI
Domain	Personal finance (India): loans, investments, savings and budgeting
Core technologies	Python 3.12, LangChain 0.3, OpenAI GPT-4o-mini, Streamlit, pytest
Source code	https://github.com/Vineeth-Muraleedharan/personal-finance-ai-agent/tree/main
Streamlit app	https://personal-finance-ai-agent-by-vineeth.streamlit.app/

1.2 Problem statement

Everyday money decisions such as taking a home loan, starting a monthly investment or planning a budget depend on arithmetic that most people find tedious and easy to get wrong. A loan EMI needs a compound-interest formula, a SIP maturity value needs a geometric series, and a savings goal needs the same formula run backwards. General-purpose chatbots can explain these ideas well, but they are unreliable at the arithmetic itself: a language model predicts text, so a long multiplication or a power such as (1.00667)^240 can come out slightly or badly wrong while still sounding confident.

This project builds an AI agent that separates the two jobs. The language model understands the question, decides what is being asked and picks the right tool. The numbers themselves are produced by deterministic Python functions. The user gets natural conversation and the accuracy of a calculator, with the tool call visible so the answer can be checked.

1.3 Why this matters

- **Accuracy:** financial answers must be exact to the rupee; the agent never estimates arithmetic.
- **Transparency:** every answer lists which tool ran, with its inputs and raw output.
- **Accessibility:** plain-English questions replace spreadsheets and separate calculator sites; Indian units such as lakh and crore are understood.
- **Agentic pattern:** the same design (LLM as planner, code as executor) applies to many other domains, so the project is a reusable template.

1.4 Objectives

- Build a LangChain agent on GPT-4o-mini that automatically selects among at least three tools.
- Implement tools for percentage and basic calculations, loan EMI, SIP future value, required monthly savings and a 50-30-20 budget.
- Provide a clean, mobile-friendly Streamlit chat interface and deploy it publicly.
- Demonstrate the agent on multiple queries and verify every result independently.
- Protect the deployment with input validation and cost controls.

2. Requirements and Scope

The assignment brief asked for a finance agent that answers questions using LangChain tools and a deployed interface. Table 1 maps each requirement to the part of the solution that meets it.

Requirement	How it is met
Use LangChain with GPT-4o-mini	OpenAI (model='gpt-4o-mini', temperature=0) inside a tool-calling agent (agent.py)
At least 3 tools	Five tools: calculator, emi_calculator, sip_calculator, required_monthly_savings, budget_planner
Agent chooses the tool automatically	Tool-calling agent; the model reads tool descriptions and selects one per question
Example: EMI for a loan	'Calculate EMI for Rs. 10 lakh at 8% for 20 years' returns Rs. 8,364.40
Example: SIP return	'Invest Rs. 7000 monthly for 15 years at 10%' returns Rs. 29,01,292
Multiple demo queries	Eight queries run by demo_queries.py and saved in demo_results.md
Deployment	Streamlit Community Cloud, with the API key held in Secrets https://personal-finance-ai-agent-by-vineeth.streamlit.app/

Table 1: Requirements traceability

2.1 In scope

- Questions about EMIs, SIP growth, savings goals, simple calculations, and budgets.
- Indian rupee formatting with lakh and crore grouping.
- Short follow-up conversation within a session.
- Public deployment with usage limits.

2.2 Out of scope

- Personalised investment advice or stock recommendations.
- Live market data, tax computation and credit scoring.
- User accounts, stored financial data, or persistent chat history.

3. System Architecture

The application has four layers. The Streamlit front end collects the question and applies input checks. The AgentExecutor runs the LangChain tool-calling loop. GPT-4o-mini decides which tool to call and writes the final wording. The tools in tools.py perform the calculation.

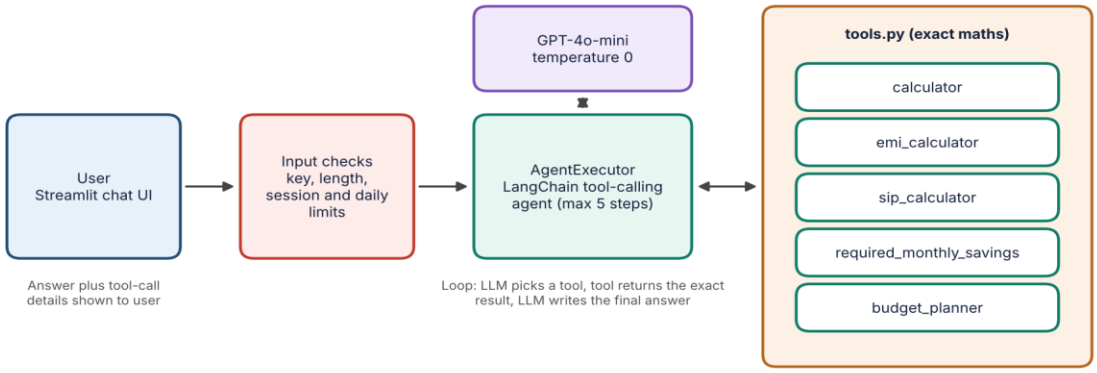


Figure 2: High-level architecture of the Personal Finance AI Agent

3.1 Request flow

1. **Question:** the user types a question or picks an example from the dropdown.
2. **Checks:** the app verifies the API key, the 300-character limit, the 15-question session limit, and the daily cap.
3. **Planning:** the model receives the system prompt, the chat history, and the tool schemas, and replies with a tool call that names a tool and its arguments.
4. **Execution:** AgentExecutor runs the named Python function and returns the exact result to the model.
5. **Answer:** the model writes a short reply using only the returned numbers; the app shows it with the tool name, inputs and output.

3.2 Project structure

The code is split by responsibility so each part can be tested on its own. The repository holds twelve files:

File	Purpose
tools.py	Five @tool functions plus pure helper functions (compute_emi, compute_sip_future_value and so on)
agent.py	System prompt, LLM factory, build_agent(), ask() returning the answer and tool calls
app.py	Streamlit interface, theme, limits and rendering of tool calls
demo_queries.py / demo_results.md	Runs eight demo questions and records the outputs
tests/	test_tools.py (13 formula tests) and test_agent_offline.py (scripted fake LLM)
assets/background.svg	Original abstract finance artwork used as the page background
.streamlit/config.toml	Dark theme colours
requirements.txt, README.md, .gitignore	Pinned dependencies, documentation, and exclusion of .env and secrets

Table 2: Repository contents

4. Tool Design

Each tool is a plain Python function decorated with @tool. The docstring is the description the model reads when choosing, so it states exactly what the tool does and what inputs it needs. Pure helper functions hold the arithmetic so that tests can call them without involving the LLM. Throughout, r is the monthly rate (annual percent divided by 12 and by 100) and n is the number of months (years times 12).

4.1 Tool summary

Tool	Purpose	Inputs
calculator	Evaluate arithmetic such as '20% of 85000'	expression (text)
emi_calculator	Monthly loan instalment, total payment and interest	principal, annual rate %, years
sip_calculator	Maturity value of a monthly investment	monthly amount, annual return %, years, optional start_of_month
required_monthly_savings	Monthly amount needed to reach a goal	target amount, annual return %, years
budget_planner	50-30-20 split of monthly income	monthly income

4.2 Formulas

```
EMI           = P * r * (1 + r)^n / ((1 + r)^n - 1)
SIP value     = M * ((1 + r)^n - 1) / r            (end of month)
              x (1 + r) for payments at start of month
Monthly goal  = FV * r / ((1 + r)^n - 1)
Budget        = 50% needs, 30% wants, 20% savings
```

The SIP tool defaults to end-of-month contributions, which is the convention used by most Indian calculators, and offers start-of-month as an option. The required-savings tool is the SIP formula solved for the monthly amount; it goes beyond the minimum brief and covers the common goal-planning question.

4.3 Safe calculator

The calculator never uses Python's eval(). It parses the expression into an abstract syntax tree and walks it, allowing only numbers, the operators + - * / ** and %, and unary signs. Any other node (a function call, a variable name, an attribute) raises an error. The phrase '20% of 85000' is rewritten to 0.2 * 85000 before parsing. This removes the code-injection risk that comes with evaluating model-written text.

4.4 Input validation and formatting

- A shared check rejects zero, negative and non-numeric values and returns a clear message, so the model can ask the user to correct the input.
- Results are formatted by an inr() helper using Indian digit grouping, for example Rs. 20,07,456.17, so the figures match what users expect.
- Tools return structured text containing every figure the answer needs (EMI, total payment, interest), so the model never has to calculate a second number.

4.5 Worked example

For the question 'Calculate EMI for Rs. 10 lakh at 8% for 20 years', the model converts the amount and calls emi_calculator(principal=1000000, annual_rate=8, years=20). The tool computes r = 8 / 12 / 100 = 0.006667 and n = 20 x 12 = 240, then applies the EMI formula. The result is returned as structured text:

```
EMI: Rs. 8,364.40 | Total payment: Rs. 20,07,456.17 | Total interest: Rs.
10,07,456.17
```

The model only rephrases this output into a sentence, so every figure in the reply can be traced to the tool result shown in the expander. A change of one percentage point in the rate (9% instead of 8%) raises the EMI to Rs. 8,997.26 and the total interest by about Rs. 1.52 lakh, which is the kind of comparison the agent answers in one step.

5. Agent Design

5.1 Model and configuration

Setting	Value and reason
Model	gpt-4o-mini: low cost, fast, reliable at tool calling
Temperature	0, so the same question gives the same tool choice and wording as far as possible
Agent type	create_tool_calling_agent with AgentExecutor (LangChain 0.3)
max_iterations	5, which stops loops and bounds cost
return_intermediate_steps	True, so the UI can show each tool call
Memory	Recent chat history passed with each question within the session

Table 4: Agent configuration

5.2 System prompt

The system prompt defines the assistant as a personal finance helper for India that answers in rupees. Its rules are the main control over behavior:

- Never perform financial arithmetic yourself; always call a tool.
- Convert lakh and crore to plain numbers before calling a tool (1 lakh = 100,000; 1 crore = 10,000,000).
- If a required input is missing, ask for it instead of guessing.
- Keep answers short and base them only on tool output.
- Add a reminder that market returns are not guaranteed, but only for investment questions.
- Politely decline requests unrelated to personal finance.

5.3 Tool-calling loop

With tool calling, the model does not write 'Action:' text for a parser to read. It returns a structured call such as emi_calculator(principal=1000000, annual_rate=8, years=20). The executor runs the function, appends the result to the conversation and calls the model again, which then writes the final reply. Most questions need one tool call and two model calls. A multi-part question can trigger several calls in a row, which the 5-step cap allows.ask() in agent.py returns the final answer together with a list of the tool calls (tool name, inputs, output) taken from the intermediate steps. The interface shows these in an expander under each answer, so a reader can see that the numbers came from code and can re-check them.

5.4 Design decisions

Decision	Reason
Tools do all arithmetic	Removes the main failure mode of language models in finance
Pure functions separate from @tool wrappers	Formulas can be unit tested without an API key
Temperature 0	Repeatable, auditable results
Pinned LangChain below 0.4	LangChain 1.x removed AgentExecutor from langchain.agents; pinning keeps the code working

6. Application Design

The interface is a single-column Streamlit chat page centred on the screen with the sidebar removed, so it works the same on a phone as on a desktop. The look is a dark finance theme with an original SVG background (candlestick chart and rings), a neon window frame and a neon prompt box.

6.1 Interface features

- **Title and caption:** 'Smart finance answers. AI that calculates, not guesses.'
- **Example dropdown:** six ready questions; choosing one runs it immediately.
- **Tools badge:** a neon '!' at the top right opens a panel listing all five tools. After an answer, the tools that were used are highlighted as USED and the others fade.
- **Chat area:** user and assistant messages as cards, with a 'Tool used' expander showing inputs and raw output.
- **Prompt box:** neon-outlined, pinned at the bottom inside the fixed neon frame, with a neon send arrow.
- **Clear chat:** resets the conversation and the tool highlights.

6.2 Safeguards and cost control

Control	Behavior
API key handling	Read from Streamlit secrets in deployment, or a local .env file; never shown to the user and never committed (.gitignore)
Input length	Questions over 300 characters are rejected
Session limit	15 questions per session
Daily cap	200 questions per day across all users, counted in a cached server-side counter
Error messages	Generic text only; no stack traces or key details reach the user
Provider-side limits	A monthly spending cap and an email alert set on the OpenAI account

Table 6: Safeguards

Limits are applied silently: no running counter is shown, and a clear message appears only when a limit is reached.

6.3 Interface evolution

The first version had a sidebar for the API key and a question counter. It would be awkward on mobile, so the key moved to secrets and the sidebar was removed. Later iterations added the example dropdown, background artwork, a window-fixed neon frame and the tools badge.

7. Testing and Evaluation

7.1 Automated tests

Fourteen pytest tests run without an API key and pass:

Test file	Tests	What is checked
tests/test_tools.py	13	EMI, SIP (end and start of month), required savings, budget split, calculator parsing, percentages, rejection of unsafe expressions, invalid inputs, Indian number formatting
tests/test_agent_offline.py	1	Full agent loop with a scripted fake LLM: tool is called, result is returned, intermediate steps are captured

Table 7: Automated tests (14 passed)

7.2 Demo queries

demo_queries.py sent eight questions to the live agent and saved the results to demo_results.md. Each result was also checked against the formulas independently (Section 7.3).

#	Question	Tool used	Result
1	EMI for Rs. 10 lakh, 8%, 20 years	emi_calculator	EMI Rs. 8,364.40; total Rs. 20,07,456.17; interest Rs. 10,07,456.17
2	Invest Rs. 7,000 monthly, 15 years, 10%	sip_calculator	Rs. 29,01,292; invested Rs. 12,60,000; gain Rs. 16,41,292
3	Budget plan for income Rs. 90,000	budget_planner	Rs. 45,000 / Rs. 27,000 / Rs. 18,000
4	What is 20% of Rs. 85,000	calculator	Rs. 17,000
5	Rs. 3,000 per month, 5 years, 8%	sip_calculator	Rs. 2,20,430.57; invested Rs. 1,80,000; gain Rs. 40,430.57
6	Rs. 5,000 per month, 10 years, 12%	sip_calculator	Rs. 11,50,193.45; invested Rs. 6,00,000; gain Rs. 5,50,193.45
7	Save monthly for Rs. 20 lakh in 10 years at 12%	required_monthly_savings	Rs. 8,694.19 per month; invested Rs. 10,43,302.76; gain Rs. 9,56,697.24
8	EMI for Rs. 10 lakh, 9%, 20 years	emi_calculator	EMI Rs. 8,997.26; total Rs. 21,59,342.29; interest Rs. 11,59,342.29

Table 8: Demo queries and results

7.3 Verification of the numbers

The headline results were recomputed separately with the closed-form formulas in Python: the EMI figures match to the paisa (Rs. 8,364.40 and Rs. 8,997.26), and the SIP values match as well (Rs. 29,01,292 for query 2 and Rs. 2,20,430.57 for query 5). Because the tools use the same formulas, the check confirms that the model passed the right inputs, converted lakh correctly and reported the tool output without change.

7.4 Observations

- The agent chose the correct tool in all eight queries without hints.
- Lakh values were converted correctly (Rs. 10 lakh became 1,000,000; Rs. 20 lakh became 2,000,000).
- Answers were short and quoted the tool figures verbatim, which is the behavior the system prompt asks for.
- The market-return reminder appeared only on investment questions, not on EMI or budget questions.

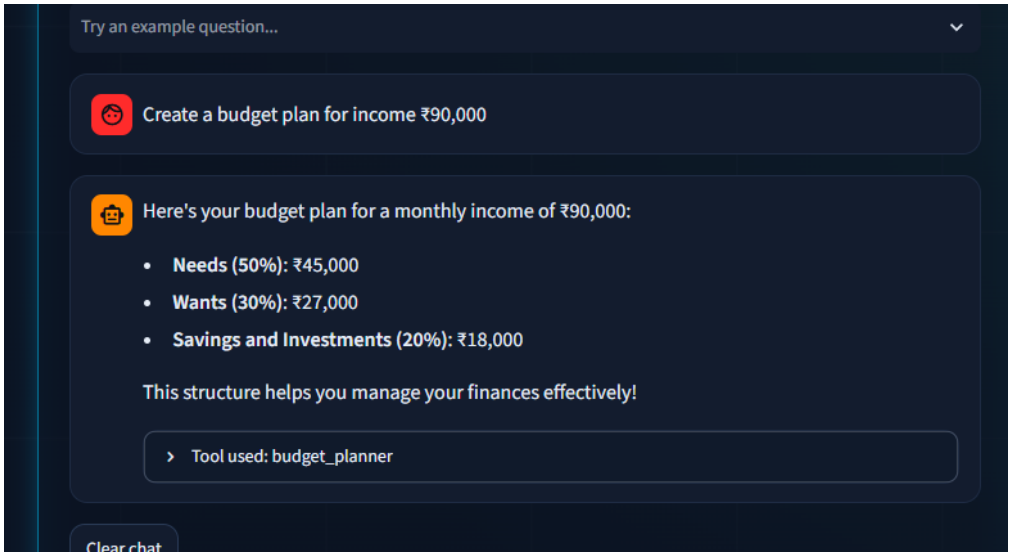


Figure 5: Budget answer with the 'Tool used: budget_planner' expander



Figure 6: EMI answer for Rs. 10 lakh at 8% for 20 years (earlier theme)

8. Deployment

8.1 Local setup

```
conda create -n pf-agent python=3.12
conda activate pf-agent
pip install -r requirements.txt
# put OPENAI_API_KEY=... in a local .env file (gitignore)
streamlit run app.py
pytest
```

8.2 Source control

The code is published in a public GitHub repository: https://github.com/Vineeth-Muraleedharan/personal-finance-ai-agent/tree/main. The .gitignore keeps .env, .streamlit/secrets.toml and cache folders out of version control, so no key is ever committed. The first push was rejected because GitHub had created a starter commit; it was resolved by pulling with unrelated histories allowed and then pushing.

8.3 Streamlit Community Cloud

- The GitHub repository was connected to Streamlit Community Cloud with app.py as the entry point.
- OPENAI_API_KEY was added under the app's Secrets, so the key lives only on the server.
- Dependencies install from the pinned requirements.txt (LangChain below 0.4, Streamlit 1.40 or later, python-dotenv).
- The app is live at the address below and redeploys automatically on every push to main.

Streamlit app: https://personal-finance-ai-agent-by-vineeth.streamlit.app/

8.4 Issues met and fixed

Issue	Fix
ImportError: AgentExecutor not in langchain.agents	Unpinned install pulled LangChain 1.x; restored pinned ranges below 0.4
ModuleNotFoundError: dotenv	Installed python-dotenv and added it to requirements.txt
API key not found in demo_queries.py	agent.py did not call load_dotenv(); added it
Neon frame moved with scrolling	Replaced with a frame fixed to the window using a .stApp::after layer
Git push rejected	Pulled the starter commit with --allow-unrelated-histories, then pushed

Table 9: Problems encountered

8.5 Operating the deployed app

After deployment, the app needs very little maintenance. These routine checks keep it reliable and inexpensive:

- **Usage and cost:** review the OpenAI usage page now and then; the monthly cap and the email alert stop any surprise bill.
- **Key rotation:** if the key is ever exposed, revoke it on the OpenAI dashboard and paste a new one into the app's Secrets; no code change is needed.

9. Limitations and Future Work

9.1 Limitations

- Fixed assumptions: constant annual return, monthly compounding, no taxes, fees, inflation or prepayment.
- Returns are illustrative; real markets are not constant, and the app is not investment advice.
- One-time EMI schedule only; no amortisation table or changing interest rates.
- No persistence: chat history is lost when the session ends.
- The agent depends on an external API, so an outage or rate limit stops answers; the daily cap also limits heavy use.

9.2 Future work

- Add tools for amortisation schedules, step-up SIPs, lump-sum investments, FD and RD maturity, and inflation-adjusted goals.
- Show charts of loan balance and SIP growth in the interface.
- Add a tax tool for Indian income-tax slabs and 80C planning.
- Add retrieval over trusted finance documents for explanatory answers, while keeping tools for arithmetic.
- Add evaluation scripts that compare the agent's answers with reference values across a larger question set.
- Support Hindi and Malayalam questions.

10. Conclusions

The project delivers a working, deployed finance agent that meets the Capstone 3 brief: LangChain with GPT-4o-mini, five tools (more than the three required), automatic tool selection, a Streamlit interface and a set of demonstrated queries. The central design choice, letting the model decide and letting code calculate, gave exact results in every test, and showing the tool call beside each answer makes the output easy to trust and check.

The work also covered practical engineering: pinned dependencies, secret handling, offline tests that need no API key, usage limits for a public link, and a mobile-friendly design. The same structure can be reused for other domains by swapping in new tools and a new system prompt.

11. References

[1] LangChain documentation: Tools and tool-calling agents. https://python.langchain.com/

[2] OpenAI API documentation: Function (tool) calling and the GPT-4o-mini model. https://platform.openai.com/docs

[3] Streamlit documentation: Chat elements, secrets management and Community Cloud deployment. https://docs.streamlit.io/

[4] Python documentation: the ast module (safe expression parsing). https://docs.python.org/3/library/ast.html

[5] pytest documentation. https://docs.pytest.org/

12. Project Milestones and Submission

Milestone	Status
Tools designed and unit tested	Done: 5 tools, 13 formula tests
Agent built with LangChain and GPT-4o-mini	Done: tool-calling agent, offline test with fake LLM
Demo queries run and verified	Done: 8 queries, results saved in demo_results.md
Streamlit interface	Done: mobile-friendly single column, neon theme, tools badge
GitHub repository	Done: https://github.com/Vineeth-Muraleedharan/personal-finance-ai-agent/tree/main
Deployment	Done: https://personal-finance-ai-agent-by-vineeth.streamlit.app/
Model Development Document	Done: this document

VINEETH M

Applied Data Science, Machine Learning and AI
E & ICT Academy, IIT Guwahati
Email: vinuvineeth616@gmail.com
Mob: +91 7560946570

GitHub: https://github.com/Vineeth-Muraleedharan/personal-finance-ai-agent/tree/main

Streamlit app: https://personal-finance-ai-agent-by-vineeth.streamlit.app/